ACA

Linux Server

Docente: Wolfgang Cecchin

Titolo argomento:

mysql (mariadb)

“Configurazione” del mysql

- Installiamo MySQL;
- Configuriamo MySQL.

Installare mysql (mariadb)

MySQL e MariaDB: differenze e similitudini

1) Origine e storia

MySQL nasce nel 1995 e diventa uno dei database relazionali open source più diffusi al mondo. Nel 2010 viene acquisito da **Oracle**, che ne mantiene lo sviluppo con una doppia licenza: community (GPL) e commerciale.

MariaDB nasce nel 2009 come **fork diretto di MySQL**, creato dagli stessi fondatori originali (Michael "Monty" Widenius e altri) proprio in risposta all'acquisizione da parte di Oracle, temendo una chiusura del progetto. Il nome deriva da Maria, figlia di Widenius (come MySQL derivava da "My", l'altra figlia).

La sintassi tra le due versioni è comune al 95%. Dalla versione 10.x di MariaDB in poi, la compatibilità con MySQL non è più garantita al 100% — alcune funzionalità avanzate di MySQL non esistono in MariaDB e viceversa.

Regola pratica: MariaDB è la scelta più comune nelle distribuzioni Debian/Ubuntu (è il pacchetto installato di default con apt install mariadb-server), mentre MySQL è più frequente in ambienti enterprise o su sistemi Red Hat/CentOS.

$ sudo apt install mariadb-server

$ sudo systemctl start mariadb

$ sudo systemctl enable mariadb

$ sudo systemctl status mariadb

Creazione utenti e creazione di un semplice database

Entrare nel mysql, utilizzando il client:

$ sudo mysql -u root -p

-u = user
- p = password

Per creare un datbase:

CREATE DATABASE miodatabase;

SHOW DATABASES;

Per usarlo:
USE miodatabase;

Per creare un utente per utilizzare il database:

CREATE USER 'its-usr'@'localhost' IDENTIFIED BY 'password';

Per attribuire i privilegi:

GRANT ALL PRIVILEGES ON miodatabase.* TO 'username'@'localhost';

FLUSH PRIVILEGES;

Forma deprecata:

GRANT ALL ON nomedb.* TO 'its-usr'@'localhost' IDENTIFIED BY 'password';
(creava utente privilege in un solo passaggio.

L'utente è attivo solo per localhost, cioè solo per la macchina locale.

Per cancellare un utente (revoca anche i privilegi):

DROP USER IF EXISTS 'its-usr'@'localhost';

Creare un utente che si connetta da qualsiasi IP (sconsigliabile)

CREATE USER 'its-usr'@'%' IDENTIFIED BY 'password';
GRANT ALL PRIVILEGES ON miodatabase.* TO 'its-usr'@'%';
FLUSH PRIVILEGES;

Attenzione!
Per accedere da remoto è anche necessario modificare il file:

/etc/mysql/mariadb.conf.d/50-server.cnf
Modificare la configurazione:

bind-address = 0.0.0.0

Avvertenze di sicurezza

- **Non usare @'%' in produzione** — preferire sempre un IP specifico o una subnet (@'192.168.1.%')
- **Non esporre la porta 3306 su Internet** — usare una VPN o un tunnel SSH
- Il bind su 0.0.0.0 espone il database su **tutte** le interfacce di rete, inclusa quella pubblica se present

Approfondimento: Apici singoli, Backtick, Virgolette doppie

' Apici singoli — stringhe e valori

Usati per delimitare valori letterali: stringhe, password, date.

SELECT * FROM utenti WHERE nome = 'Mario';

INSERT INTO prodotti (nome) VALUES ('Laptop');

CREATE USER 'its-usr'@'localhost' IDENTIFIED BY 'password';

` Backtick — identificatory
(Linux / Windows: Altgr + ì; oppure Windows: Alt + 96 su tastierino numerico)

Usati per delimitare nomi di oggetti del database: tabelle, colonne, database. Necessari quando il nome:

- coincide con una parola riservata di SQL
- contiene spazi o caratteri speciali

```
-- "select" è una parola riservata: senza backtick darebbe errore
SELECT `select`, `order` FROM `my-table`;

-- Normalmente non servono, ma non fanno mai male
SELECT `nome`, `cognome` FROM `utenti`;
```

I backtick servono solo in casi particolari. Se i nomi delle tabelle e colonne sono normali (niente spazi, niente parole riservate), puoi scrivere senza.

```
-- Questi sono equivalenti se "utenti" non è una parola riservata:
SELECT nome FROM utenti;
SELECT `nome` FROM `utenti`;
```

" " Virgolette doppie

Hanno un comportamento variabile a seconda delle configurazioni del database.

Esportare databases

```
mysqldump -u root -p nome_database > backup.sql

# Include la creazione del database stesso
mysqldump -u root -p --databases nome_database > backup.sql

# Dump di più database
mysqldump -u root -p --databases db1 db2 db3 > backup.sql

# Dump di tutti i database
mysqldump -u root -p --all-databases > backup_completo.sql
```

Importare databases

mysql -u root -p nome_database < backup.sql

Se il dump è stato fatto con --databases (include il CREATE DATABASE):

Non serve specificare il database, è già nel file

mysql -u root -p < backup.sql